Introduction to Object-Oriented Programming

What is OOP?

Object-Oriented Programming (OOP) is a programming paradigm that organises software design around **objects** rather than functions and logic. An object is an entity that contains both data (called data members or attributes) and the functions that operate on that data (called member functions or methods).

OOP is built on the idea of modelling programs after real-world entities. For example, a "Car" object might have attributes like colour and speed, and methods like accelerate() and brake().

OOP vs Procedure-Oriented Programming

Feature	Procedure-Oriented (C)	Object-Oriented (C++)
Organisation	Divided into functions	Divided into objects
Focus	Functions and sequence of actions	Data and the objects that hold it
Approach	Top-down	Bottom-up
Data access	Open — any function can access data	Controlled via access specifiers (public, private, protected)
Data security	Low — no data hiding	High — data hiding via encapsulation
Overloading	Not supported	Function and operator overloading supported
Reusability	Limited	High — through inheritance and classes

Key Principles of OOP

The six main principles of OOP are:

- **Encapsulation** — wrapping data and functions together in a class, hiding internal details
- **Data Abstraction** — exposing only essential features, hiding background complexity
- **Polymorphism** — one name, many forms; same function behaves differently depending on context
- **Inheritance** — a new class acquires properties and behaviours of an existing class
- **Dynamic Binding** — the function to be called is determined at runtime, not compile time
- **Message Passing** — objects communicate by calling each other's member functions

Benefits of OOP

- **Reusability**: Classes and functions can be reused across programs without modification
- **Reduced complexity**: A problem is broken into objects, each responsible for a specific task
- **Data hiding**: Private data cannot be accessed directly from outside the class, improving security
- **Easy maintenance**: Changes inside a class do not affect other parts of the program
- **Extensibility**: New objects can be created with small differences to existing ones using inheritance
- **Modularity**: Each object is a self-contained unit, making large programs easier to manage